

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#01)

Apply modern simulation tools ModelSim and Logisim to implement basic processor components

Student Name: Huzaifa Kashif Roll No: 24I-6501 Section: CE-A

Date performed: 26th September , 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Prepared by:

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

Used Basic Verilog commands and specifically gate level instructions to build an and,or, not gate.

Code:

```
module AndGate(A,B,cout);
input A;
input B;
output cout;
and a1(cout,A,B);
endmodule

module OrGate(A,B,cout);
input A;
input B;
output cout;
or a1(cout,A,B);
endmodule

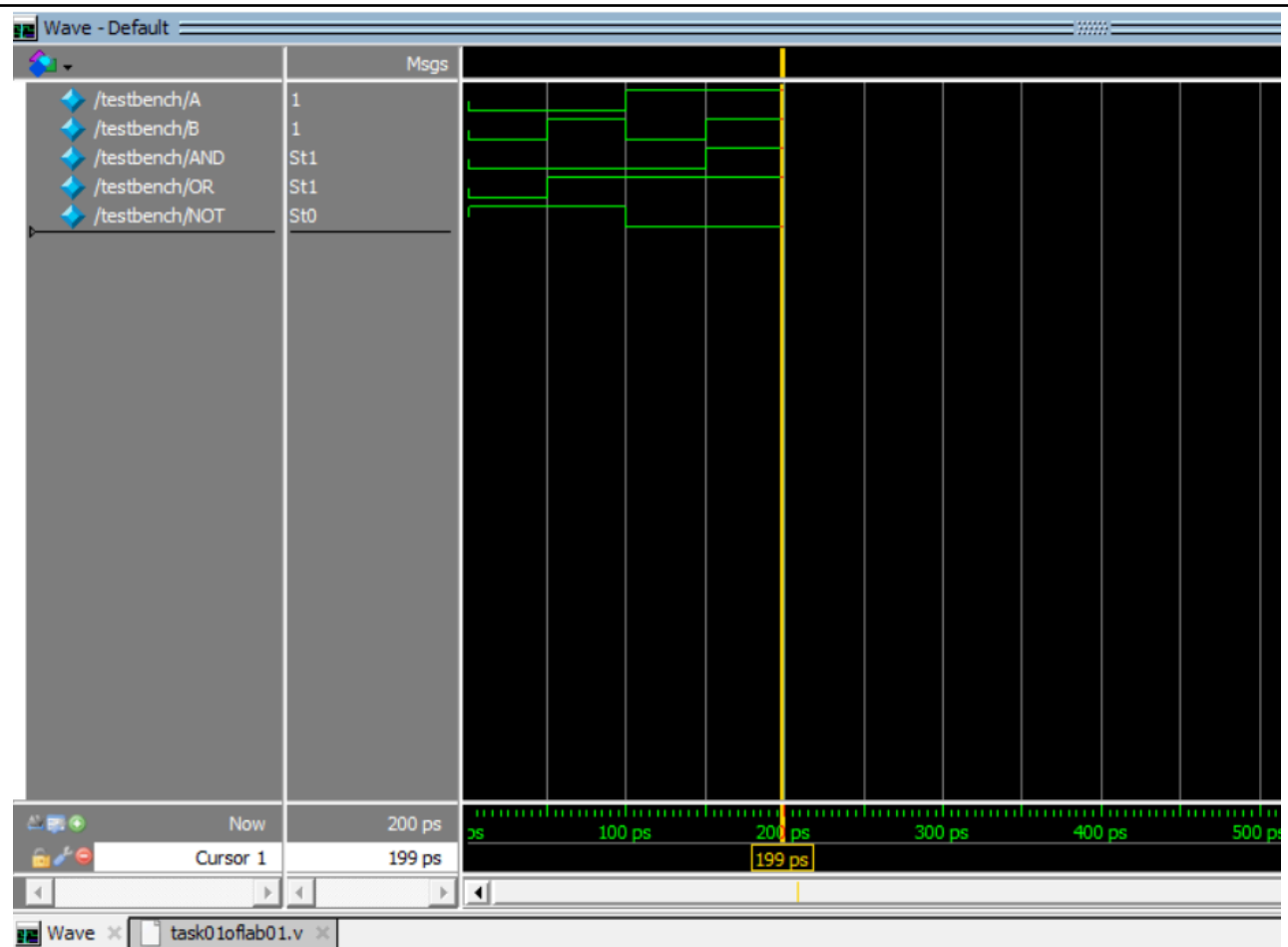
module NotGate(A,cout);
input A;
output cout;
not a1(cout,A);
endmodule

module testbench();
reg A,B;
wire AND,OR,NOT;
AndGate uut(A,B,AND);
OrGate uup (A,B,OR);
NotGate abc (A,NOT);
initial

begin
A=0; B=0;
#50 A=0; B=1;
#50 A=1; B=0;
#50 A=1; B=1;
end
```

endmodule

Output:



Task 2:

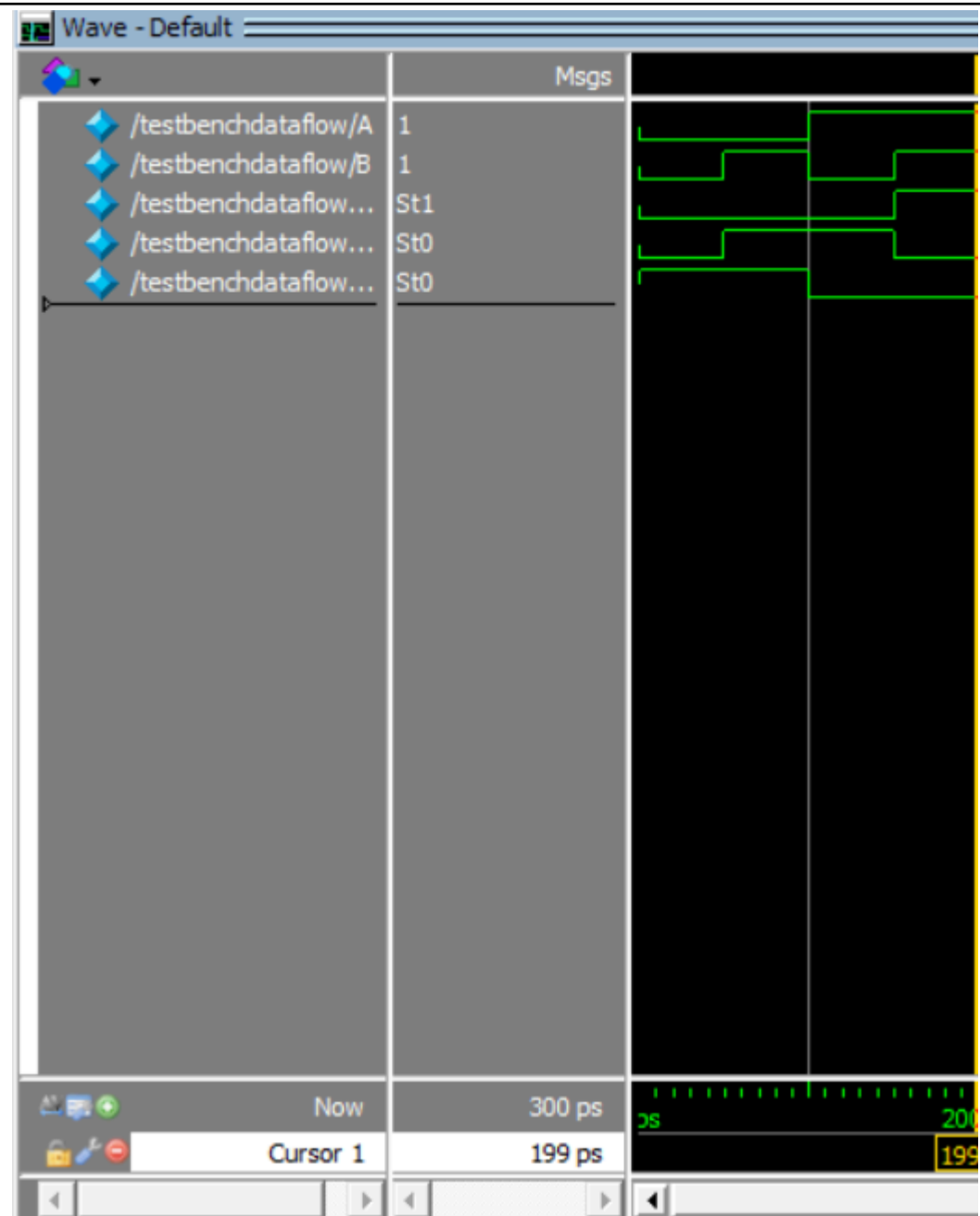
Description:

Similarly built and,or,not gate using varilog commands but in a data flow level approach.

Code:

```
module AndGate(A,B,andout);  
input A;  
input B;  
output andout;  
assign andout=A&B;  
endmodule  
  
module OrGate(A,B,orout);  
input A;  
input B;  
output orout;  
assign orout=A+B;  
endmodule  
  
module NotGate(A,notout);  
input A;  
output notout;  
assign notout=~A;  
endmodule  
  
module testbenchdataflow();  
reg A,B;  
wire AND,OR,NOT;  
AndGate uut(A,B,AND);  
OrGate uup (A,B,OR);  
NotGate abc (A,NOT);  
initial  
  
begin  
A=0; B=0;  
#50 A=0; B=1;  
#50 A=1; B=0;  
#50 A=1; B=1;  
end  
endmodule
```

Output:



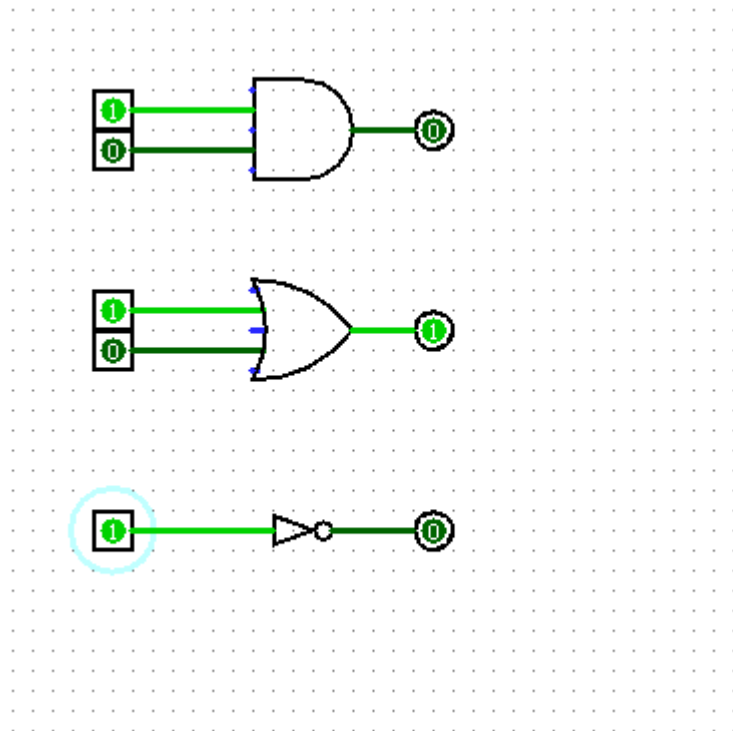
--

Task 3:

Description:

Used LogicSim to build a simple circuit to show the way logic gates work.

Output:



Task 4 (GATE LEVEL):

Description:

Here used gate level designing and data flow level designing to construct circuit. The idea here was defining wires, and adding the sub result into another gate.

Code:**GATE LEVEL:**

```
module AndGate(A,B,argout);
input A;
input B;
output argout;
and a1(argout,A,B);
endmodule

module OrGate(A,B,orout);
input A;
input B;
output orout;
or a1(orout,A,B);
endmodule

module NotGate(A,notout);
input A;
output notout;
not a1(notout,A);
endmodule

module testbenchTask4GATELEVEL();
reg A,B;
wire ANDx,ANDy,OR,ANOT,BNOT;
NotGate anot (A,ANOT);
NotGate bnot (B,BNOT);
AndGate aNOTb(ANOT,B,ANDx);
AndGate AbNOT (A,BNOT,ANDy);
```

```
OrGate ORresult (ANDx,ANDy,OR);
```

```
initial
```

```
begin
```

```
A=0; B=0;
```

```
#50 A=0; B=1;
```

```
#50 A=1; B=0;
```

```
#50 A=1; B=1;
```

```
end
```

```
endmodule
```

DATA FLOW LEVEL:

```
module AndGate(A,B,andout);
```

```
input A;
```

```
input B;
```

```
output andout;
```

```
assign andout=A&B;
```

```
endmodule
```

```
module OrGate(A,B,orout);
```

```
input A;
```

```
input B;
```

```
output orout;
```

```
assign orout=A+B;
```

```
endmodule
```

```
module NotGate(A,notout);
```

```
input A;
```

```
output notout;
```

```
assign notout=~A;
```

```
endmodule
```

```
module testbenchTask4DATALEVEL();
```

```
reg A,B;
```

```
wire ANDx,ANDy,OR,ANOT,BNOT;
```

```
NotGate anot (A,ANOT);
```

```
NotGate bnot (B,BNOT);
```

```
AndGate aNOTb(ANOT,B,ANDx);
```

```
AndGate AbNOT (A,BNOT,ANDy);
```

```
OrGate ORresult (ANDx,ANDy,OR);
```

```
initial
```

```
begin
```

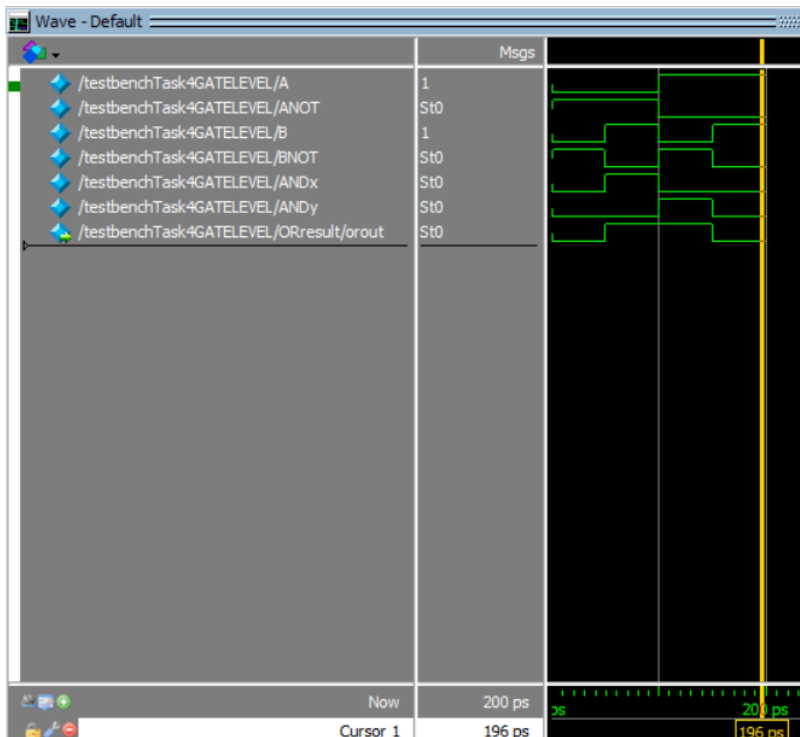
```
A=0; B=0;
```

(EL2012)

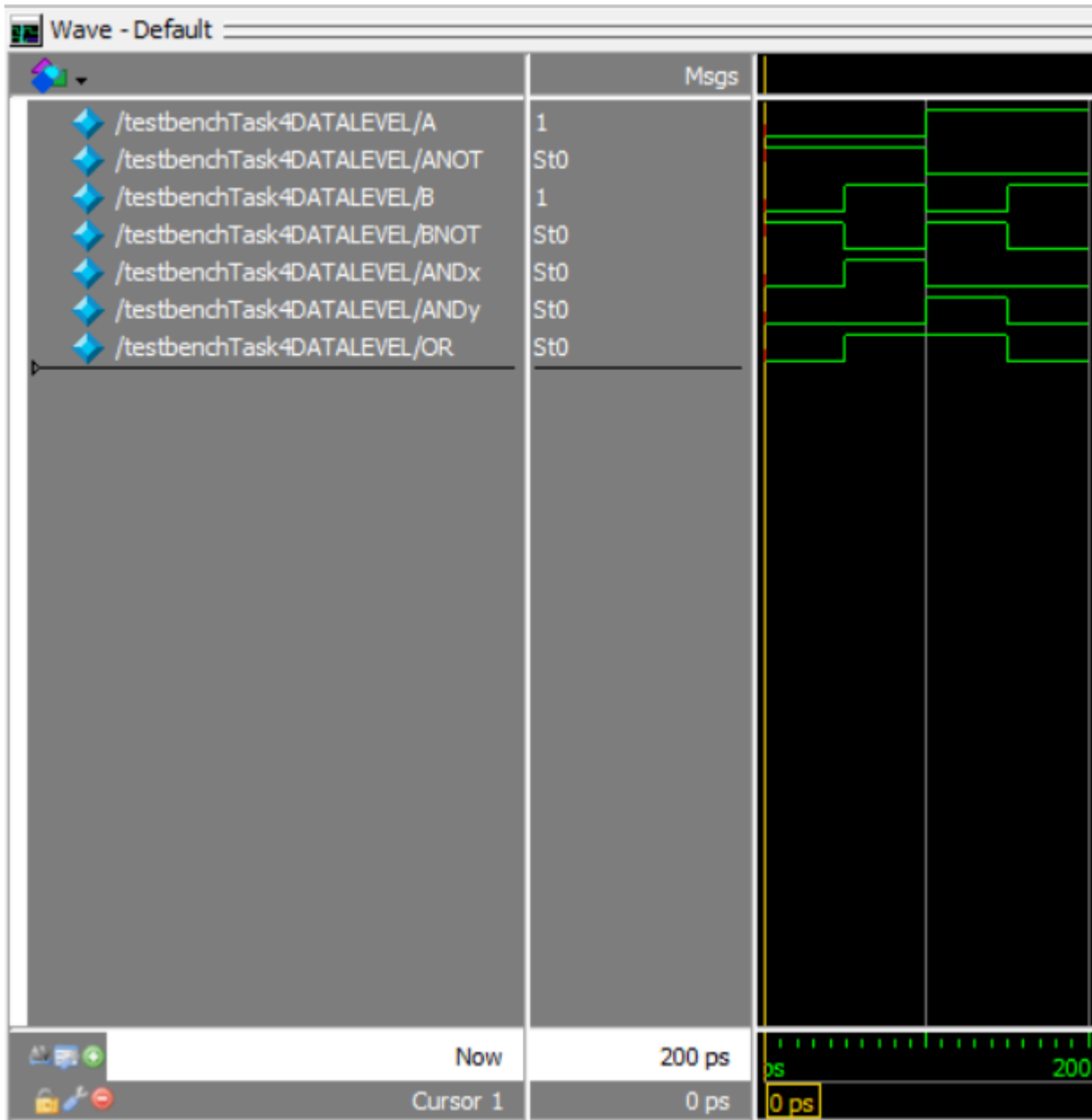
```
#50 A=0; B=1;
#50 A=1; B=0;
#50 A=1; B=1;
end
endmodule
```

Output:

GATE LEVEL:



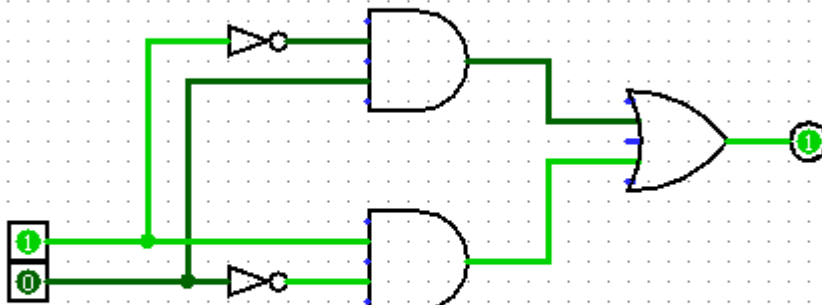
DATA FLOW LEVEL:



Task 5:

Description:

Constructed circuit on LogicSim.

Output:

Task 6:

Description:

Here used Gate Level and Data Flow designing to calculate the compliments of 4 bit inputs.

Code:

GATE LEVEL:

```
module compliment(A,Aout);

input [3:0] A;
output [3:0] Aout;

not(Aout[0],A[0]);
not(Aout[1],A[1]);
not(Aout[2],A[2]);
not(Aout[3],A[3]);

endmodule

module testbenchtask6GATELEVEL();

reg [3:0] A;
wire [3:0] Aout;

compliment G1(A,Aout);

initial
begin
    A = 4'b0000;
    #50 A = 4'b1001;
    #50 A = 4'b1011;
    #50 A = 4'b1111;
end

Endmodule
```

DATA FLOW LEVEL:

```
module compliment(inputA,outputA);

input [3:0] inputA;
output [3:0] outputA;

assign outputA[0] = ~inputA[0];
assign outputA[1] = ~inputA[1];
assign outputA[2] = ~inputA[2];
assign outputA[3] = ~inputA[3];
```

endmodule

Output:

GATE LEVEL:

Wave - Default		
	Msgs	
+ /testbenchtask6GA...	1111	0000 1001 1011 1111
+ /testbenchtask6GA...	0000	1111 0110 0100 0000

DATA FLOW LEVEL:

Wave - Default		
	Msgs	
+ /testbenchtask6DA...	1111	0000 1001 1011 1111
+ /testbenchtask6DA...	0000	1111 0110 0100 0000